

4x05 — Attack Reconstruction

HEALTHBANE Campaign: Study Guide

A task-by-task preparation guide for Tasks 0, 1, 2, 3, 4, 5, 7, 8, 9, 12 and 15. This guide explains what each exercise is asking for, what skill it is meant to build, and how to approach it — including specific *jq* / *grep* / *date* / *diff* patterns to reach for. It does not solve the exercises: the actual IOCs, timestamps and findings in your evidence files are yours to discover.

Before You Start: What This Project Actually Is

Every project in this course so far investigated ONE evidence domain at a time: 4x00 read emails, 4x01 read packet captures, 4x02 read intelligence reports, 4x03 read malware samples, 4x04 read SIEM logs. Each one taught you how to be rigorous *within* a single kind of evidence. 4x05 is different in kind, not just in scope: it hands you the outputs of all five of those investigations plus four brand-new incident-response evidence files (memory capture, disk image, 14 days of firewall logs, and the IR team's raw notes) and asks you to produce one coherent, cited, chronological account of the whole attack.

The skill being tested is not “can you run jq against a new file.” It is **cross-evidence correlation**: can you take a claim from one source, find (or fail to find) corroboration for it in a completely different source, and reason honestly about what that agreement or disagreement means? That skill is what separates a junior analyst who produces a pile of findings from a senior analyst who produces understanding.

Four ideas you need before writing a single line of code

- **CONFIRMED / PROBABLE / POSSIBLE.** This is not a vibe scale, it's a rule: CONFIRMED means direct evidence from *at least two independent sources*. PROBABLE means strong evidence from one source plus supporting context (e.g. the timing fits a known pattern). POSSIBLE means the attacker's known playbook makes this plausible, but you can't point to a specific artifact proving it happened. Every claim you write in Tasks 4, 5 and 7 needs one of these three labels, and you need to be able to defend why you picked that one and not a higher one.
- **CONVERGED / SINGLE-SOURCE / CONFLICTED.** This is the IOC/evidence-level version of the same idea (used in Task 4). An indicator that shows up in two unrelated collection methods (say, a PCAP-derived finding from 4x01 AND a firewall log from IR) is much stronger evidence than the same indicator appearing once. When two sources disagree outright, that's CONFLICTED and it needs to be resolved, not averaged away.
- **Temporal anchors.** Not every timestamp in your evidence is equally trustworthy. A temporal anchor is a timestamp you are highly confident in (e.g. an email header's Received: timestamp, or a disk \$MFT creation time) that you use to sanity-check or order everything else. When two sources disagree on a time, don't just pick the prettier number — figure out which collection method is closer to the actual event (network capture point vs. log aggregation delay vs. clock drift) and say so.
- **Citations are not optional decoration.** The project requirements are explicit: source file, timestamp, host, user account, for every claim. An uncited claim isn't a finding, it's speculation. Get in the habit now, in Task 0, of writing down exactly which file and which field you pulled a fact from — it will save you enormously by the time you reach Tasks 5 and 7, where you are citing five or six sources per paragraph.

Your toolkit, and when you'll actually use each piece

Tool	What it's for here
jq	Every evidence file that's JSON (firewall sessions, 4x02 ATT&CK mapping, IOC master, the 80%-coverage layer) gets queried with jq. Know select(), test() for regex matching, group_by(), and the date functions (strptime / mktime / strftime) for timestamp math.
grep -E / -i	Pulling structured fields out of the plain-text reports (memory_artifacts.txt, disk_forensics_report.txt, the previous_findings/*.txt summaries). These are semi-structured text, not JSON — grep with good regex is your primary tool here.
sort / uniq -c	Building the frequency counts and “top N” tables the expected outputs ask for (top destinations by bytes, most-executed prefetch entries, etc.).

date -d ... +%s	Converting timestamps to epoch seconds so you can do arithmetic on them — this is how you compute the clock-skew deltas and dwell-time windows Tasks 4, 5 and 7 explicitly ask for. Remember the timezone lesson from 4x04: always know which timezone each source's timestamps are actually in before comparing them.
diff	Directly comparing two derived lists — e.g. the IOC list implied by 4x02's ATT&CK mapping against the IOC list in reference/healthbane_ioc_master.json — to mechanically surface what changed, rather than eyeballing two long files side by side.
wc -l	Quick sanity counts (how many sessions, how many recovered files) before you trust a more complex query's output.

What this exercise is

Before you analyze a single artifact, you catalog the entire evidence landscape: every file in `ir_evidence/`, `previous_findings/` and `reference/`, what kind of evidence each one is, what time period it covers, and how much you should trust it. This is deliberately NOT analysis — it's reconnaissance of your own evidence, the same discipline as `3-data_recon.sh` back in 4x04, but one level up: instead of profiling one SIEM export, you're profiling eleven different sources of wildly different types and reliability.

What you should learn

- Why starting with analysis before inventory produces **availability bias** — you end up overweighting whatever evidence you happened to open first.
- How to assign a reliability tier (HIGH / MEDIUM / LOW) honestly: raw IR evidence (memory, disk, firewall logs) collected under controlled chain-of-custody is HIGH; a previous project's *summary* of its own raw evidence is MEDIUM (you're trusting someone else's interpretation, not the original packets/logs); the IR team's own preliminary notes are explicitly LOW until you personally verify them against the harder evidence.
- How a coverage matrix (evidence type × time period) turns a vague feeling of “we probably have gaps” into a specific, defensible list of gaps — which becomes your “critical questions” section.

How to approach it

- Don't write this as one big `jq/grep` pipeline. This task is mostly about reading carefully and transcribing structured facts — treat each source file like a witness you're interviewing once, write down name / type / date-range / reliability / key-content for each, then move on.
- For the plain-text sources, skim for anything that looks like a date, a date range, or a “period covered” statement near the top of the file — investigation summaries usually state their own scope explicitly.
- For the JSON sources (firewall logs, IOC master, ATT&CK layer), pull the actual min/max timestamp with `jq` rather than guessing from the filename or a vague description.
- Build the coverage matrix only AFTER you have all eleven entries catalogued — lay the date ranges out side by side and the gaps will be visually obvious, exactly like the ASCII bar chart in the expected output.

Commands and patterns to reach for

```
# Get the real min/max timestamp out of a JSON evidence file (don't eyeball it):
jq -r '.[].timestamp' ir_evidence/firewall_sessions_ws_recv_03.json | sort | head -1
jq -r '.[].timestamp' ir_evidence/firewall_sessions_ws_recv_03.json | sort | tail -1

# Pull a stated date / scope line out of a plain-text summary:
grep -iE 'date|period|coverage|window|scope' previous_findings/4x00_phishing_summary.txt

# Count entries in a JSON array quickly, as a sanity check before trusting a file:
jq 'length' reference/healthbane_ioc_master.json
```

Watch out for

Resist the urge to write your “critical questions” section generically (“what else don't we know?”). Each question should trace back to a specific gap you identified in the coverage matrix — a time period with no evidence, or an attack phase with only one supporting source. If you can't point to the gap that generated the question, it's not a question this inventory earned you.

What this exercise is

You're parsing one file — `ir_evidence/memory_artifacts.txt` — which contains a point-in-time snapshot of WS-RECV-03's RAM: the process list at capture, active network connections, loaded DLLs/modules, and registry hive extracts (including scheduled tasks). The conceptual hook the task gives you is important: memory shows what was *running*, which is a fundamentally different kind of evidence from disk (what was *stored*) or network capture (what was *transmitted*). A process that already deleted its own binary from disk is still sitting in this file.

What you should learn

- Why memory forensics catches things disk and network evidence structurally cannot — and conversely, why memory is incomplete (it's a snapshot at ONE moment, not a history).
- How to do IOC status triage: for every artifact you find, is it KNOWN (already in `reference/healthbane_ioc_master.json` from an earlier project), NEW (never seen before), or MODIFIED (a variant — e.g. same tool family, different filename or hash)? This is the same discipline 4x02 taught you for intelligence indicators, applied to live-system artifacts instead.
- How a scheduled task definition maps to T1053.005 and why finding one here is significant: your 4x04 hunt (Task 11 in that project) explicitly searched for scheduled-task evidence and found none in the SIEM export — so if memory shows one existed, that's a real detection gap your hunt couldn't have closed with the data it had.

How to approach it

- Work section by section through the file in the order the task lists them: processes, then network connections, then modules, then the registry/scheduled-task extract. Don't try to do all four in one pass.
- For process triage, your filter isn't just “does the name look weird” — cross-check path (is it running from a standard system directory or from `Temp/Public/ProgramData`, which earlier projects already flagged as suspicious staging locations?), and whether it's digitally signed if that information is present in the artifact.
- For network connections, you have two independent things to check each one against: the known C2 infrastructure in the IOC master JSON, AND specifically the unidentified IP James Chen flagged — treat that IP as its own mini-investigation, not just another row in the table.
- For the scheduled task, extract every field the expected output asks for (name, trigger, action/command, creation timestamp) as literal strings you can cite later — Task 7 will need to slot this exact event into the larger timeline.

Commands and patterns to reach for

```
# Cross-reference a candidate indicator (process name, IP, hash) against the IOC master:
jq -e --arg val "CANDIDATE_VALUE" '.. | select(. == $val)?'
reference/healthbane_ioc_master.json
# (exits 0 / prints a match if KNOWN; prints nothing if NEW — adjust the jq path to
# match however the IOC master actually structures its arrays, e.g. .domains[], .ips[])

# Pull everything in the memory file that looks like a network connection line:
grep -A4 -iE 'connection|ESTAB|established' ir_evidence/memory_artifacts.txt

# Isolate just the scheduled-task / registry block for close reading:
grep -n -iE 'scheduled task|TaskCache|trigger|action' ir_evidence/memory_artifacts.txt
```

Watch out for

A process running from C:\Windows\System32\ with a name like svchost.exe is not automatically legitimate — it's a classic masquerading target (you flagged exactly this risk, T1036, back in 4x02's ATT&CK mapping). Check the full path and parent process, not just the filename, before you label anything LEGITIMATE.

What this exercise is

You're parsing `ir_evidence/disk_forensics_report.txt`, which captures *history* rather than a single moment: recovered deleted files, Prefetch execution records, the full scheduled-task XML (confirming Task 1), registry Run-key persistence, an NTFS \$MFT timeline for the attack window, and anti-forensics indicators. The single highest-stakes finding in this task is the recovered deleted archives — the task is explicit that confirming data staging changes this from “security incident” to “likely-reportable data breach.” Treat that section with the care it deserves.

What you should learn

- The practical difference between **access** and **staging**: an attacker who merely connected to a database server is a very different finding from an attacker who ran queries, wrote the results to disk, and compressed them — and this distinction has direct regulatory consequences (see the HIPAA note in the project's resources).
- How Prefetch evidence answers a question memory and network evidence can't: a full *execution history* of what ran on this host, with first/last-run timestamps, even for programs no longer present on disk.
- How to reason about anti-forensics: a short, surgical gap in the Security event log is a very different (and more sophisticated) finding than a wholesale “Log Cleared” event — the absence of Event ID 1102 next to a small timestamp gap is itself a clue about attacker skill level.

How to approach it

- Pull the recovered-files table first and reason about it in three parts: naming pattern (does it look like structured data export?), size (does a .csv size look consistent with query output, and do .zip sizes look like compressed versions of it?), and chronological order (was the data pulled BEFORE or AFTER compression — this tells you the staging workflow).
- For Prefetch, your filter is role-based: WS-RECV-03 is a Records-department intake workstation. Check `reference/network_topology.txt` (or the 4x04 admin-schedule context you already know) for what this host's NORMAL software footprint should be, then flag anything in the Prefetch list that doesn't belong on that kind of machine at all.
- Build the NTFS timeline as its own deliverable — every timestamped event you find anywhere else in this report (task creation, file creation, deletion) belongs in one ordered list. This becomes a direct input to Task 7's lateral-movement/staging reconstruction, so get the ordering right now.
- For anti-forensics, don't just report “a gap exists” — note its duration and whether Event 1102 (full log clear) is present or absent, since that distinction is exactly what separates T1070.001 selective deletion from a blunter, noisier wipe.

Commands and patterns to reach for

```
# Pull just the recovered-deleted-files block:
grep -A20 -i 'recovered.*deleted\|deleted files' ir_evidence/disk_forensics_report.txt

# Extract Prefetch rows and sort by first-execution time to see what ran earliest:
grep -iE '\.exe' ir_evidence/disk_forensics_report.txt | sort -t' ' -k2

# Build your own NTFS-style ordered timeline from every timestamp in the file:
grep -oE '[0-9]{4}-[0-9]{2}-[0-9]{2}([ T][0-9]{2}:[0-9]{2}(:[0-9]{2})?)?' \
ir_evidence/disk_forensics_report.txt | sort -u
```

Watch out for

Don't conflate "staging occurred" with "exfiltration occurred." This task only has disk evidence — it can tell you data was compressed and placed in a staging directory, and even that the staged files were later deleted. It cannot, by itself, tell you whether those bytes ever left the network. That specific question is deliberately deferred to Task 3 (firewall analysis), which is the only evidence source that can actually confirm or rule out exfiltration. Keep your Task 2 conclusion scoped to what disk evidence can prove.

What this exercise is

ir_evidence/firewall_sessions_ws_recv_03.json is a JSON array of stateful session records (source, destination, port, protocol, bytes transferred, duration, timestamps) covering the FULL 14-day window — the first evidence source in this whole reconstruction that does that. Your 4x01 PCAP coverage was only 48 hours; this file is the one piece of evidence positioned to answer the exfiltration question definitively, because session-metadata byte counts are exactly what you need to see data leaving the network, even without packet content.

What you should learn

- Why session-log metadata (bytes transferred, duration) can answer questions packet captures sometimes can't, simply because of collection-window length — breadth-of-time vs. depth-of-content is a real, recurring tradeoff in evidence collection.
- How to characterize an unknown IP methodically: session count, total bytes, port/protocol, and specifically its TIME PATTERN (fixed interval? off-hours only?) — this is literally the same beaconing-interval analysis you already built detection logic for in 4x01 and 4x04, now applied to a brand-new indicator.
- How to directly test an exfiltration hypothesis with arithmetic: does the total suspicious outbound byte count exceed, roughly match, or fall short of the staging file sizes you measured in Task 2? That comparison IS the exfiltration finding — it's not a qualitative judgment call, it's a number you can compute.

How to approach it

- Start with the overall internal-vs-external breakdown before chasing the unknown IP — you need the full-population baseline (how much data normally moves, how many sessions per day) before a specific IP's numbers mean anything in context.
- Build the unknown-IP profile as its own mini-report with every field the expected output lists: first/last seen, session count, interval pattern, bytes in/out. Then explicitly cross-check the port and protocol against what 4x01 documented for the known C2 channel — similarity (or lack of it) is your main evidence for the secondary-C2-vs-unrelated-traffic judgment call.
- For the temporal analysis, bucket sessions by hour of day across all 14 days, then separately pull just the off-hours EXTERNAL sessions and compare their dates against the specific dates your 4x04 hunt already flagged as lateral-movement nights — alignment here is strong corroborating evidence.
- Save the exfiltration-threshold comparison for last, after you have both numbers (suspicious outbound bytes vs. Task 2's staging sizes) solidly computed — don't eyeball this comparison, compute both sides precisely and state the delta.

Commands and patterns to reach for

```
# Internal vs external session counts and total bytes:
jq '[.[] | select(.dest_ip | test("^10\\."))] | length'
ir_evidence/firewall_sessions_ws_recv_03.json
jq '[.[] | select(.dest_ip | test("^10\\.")) | not] | '\
'length=\(length) bytes=\([.[]|.bytes_out] | add)'\ \
ir_evidence/firewall_sessions_ws_recv_03.json
```

```
# Top external destinations by total bytes (adjust field names to match your file):
jq -r '[.[] | select(.dest_ip | test("^10\\.")) | not] | '\
```

```
'[.dest_ip, .dest_port, .bytes_out] | @tsv' ir_evidence/firewall_sessions_ws_recv_03.json \  
| awk '{sum[$1" "$2]+=$3} END {for (k in sum) print sum[k], k}' | sort -rn | head -10  
  
# Epoch-second math for interval analysis (fixed-interval beaconing check):  
jq -r --arg ip "UNKNOWN_IP" '.[ ] | select(.dest_ip==$ip) | .timestamp' \  
ir_evidence/firewall_sessions_ws_recv_03.json | while read -r ts; do date -d "$ts" +%s; done |  
sort -n
```

Watch out for

Check the actual field names in your firewall JSON before copying any jq pattern literally — the examples above assume plausible field names (dest_ip, bytes_out, timestamp); run `jq '.[0]'` on the file first to see the real schema. Also remember the 4x04 lesson about firewall vs. network-capture timestamps representing different moments (connection-start vs. mid-session capture) — expect a small, explainable delta when you cross-check this file's timestamps against 4x01's, not an exact match.

What this exercise is

This is the analytical hinge of the whole project. You take everything Tasks 0–3 produced, plus every previous_findings/ summary, and build three matrices: an IOC correlation matrix (which sources contain which indicators), a timeline correlation matrix (which sources support which key events, and where they disagree), and a technique correlation matrix (which ATT&CK techniques are supported by which sources, and whether 4x02's original INFERRED calls can now be upgraded or must be corrected). This is where five separate investigations stop being five separate investigations.

What you should learn

- How to formally classify evidence convergence instead of describing it in prose: every unique IOC gets exactly one label — CONVERGED, SINGLE-SOURCE, or CONFLICTED — based strictly on how many independent sources contain it.
- How to resolve a timestamp contradiction properly: document BOTH values, propose a specific mechanical cause (timezone offset, clock skew, collection-point delay, aggregation lag), and then explicitly state which source you treat as authoritative going forward and why — “I picked the bigger number” is not a resolution.
- Why a technique's status can change in three different directions once more evidence arrives: INFERRED → CONFIRMED (new direct evidence appeared), stays INFERRED (no new evidence either way), or — the one people forget is possible — CORRECTED (the new evidence shows the original inference was specifically wrong, not just unconfirmed).

How to approach it

- Build the IOC matrix as a literal table: one row per unique indicator, one column per source (4x00 through IR), mark presence. Don't try to hold this in your head — a spreadsheet-shaped jq/awk pipeline that emits exactly this table will save you from missing a convergence that's scattered across two files you read an hour apart.
- Only AFTER the IOC matrix is built, scan it for rows with 2+ marks (your CONVERGED list) and rows that look similar but don't exactly match (your CONFLICTED candidates — e.g. the same C2 IP reported with a slightly different port or hostname in two sources).
- For the timeline matrix, pick your temporal anchors first (the handful of events you're most confident about, usually the ones with the most independent corroboration), then work outward, ordering less-certain events relative to those anchors rather than trying to pin every event to an absolute timestamp independently.
- For the technique matrix, go through 4x02's ATT&CK mapping (reference/attck_navigator_80pct.json, representing the post-4x04 state) technique by technique and ask, for each one: did Tasks 0–3 or the previous findings add NEW direct evidence for this specific technique? That's a mechanical pass, not a creative one — resist the urge to upgrade a technique's confidence without a specific artifact behind the upgrade.

Commands and patterns to reach for

```
# Quick diff-based check: which IOCs are in the new IR firewall data but NOT in the
# existing IOC master (a fast way to seed your "NEW IOCs from IR" list):
jq -r '.[].dest_ip' ir_evidence/firewall_sessions_ws_recv_03.json | sort -u > /tmp/fw_ips.txt
jq -r '.. | strings | select(test("^[0-9]{1,3}\\.{3}[0-9]{1,3}$"))' \
reference/healthbane_ioc_master.json | sort -u > /tmp/known_ips.txt
diff /tmp/known_ips.txt /tmp/fw_ips.txt # lines starting with '>' are new to IR

# Epoch-second delta between two timestamps from two different sources, for
```

```
# resolving a timeline contradiction:  
echo $(( $(date -d "SOURCE_A_TIMESTAMP" +%s) - $(date -d "SOURCE_B_TIMESTAMP" +%s) ))
```

Watch out for

A CONFLICTED finding is a result, not a failure — don't quietly drop a contradiction because resolving it is harder than reporting a clean convergence. The project requirements are explicit that contradictions must be resolved, not ignored, and an unresolved (or hidden) contradiction is exactly the kind of thing that undermines a report's credibility with the board James Chen described.

What this exercise is

The first narrative-writing task. Using your Task 4 correlation matrix as the evidentiary backbone, you reconstruct Stage 1 (phishing delivery through credential theft) and Stage 2 (C2 channel establishment) as a cited, chronological account — not a summary of the 4x00/4x01 reports, but a new synthesis that pulls in the firewall evidence those earlier projects never had access to.

What you should learn

- How to write a reconstruction paragraph correctly: event → evidence citation(s) → ATT&CK technique → confidence level, every time, with no exceptions for “obvious” claims.
- How a single new evidence source (14-day firewall logs) can re-date a finding from an earlier project — specifically, determining whether the secondary C2 IP from Task 3 was already active during this early stage, or only appeared later, changes the story about when the attacker actually deployed their backup infrastructure.
- How to write a PROBABLE finding honestly: the secondary C2 channel, by construction, only has ONE source (the firewall logs — 4x01's PCAP window ended before it appeared). That's a textbook PROBABLE, not CONFIRMED, and the exercise is explicitly testing whether you resist the temptation to round it up.

How to approach it

- Write Stage 1 first, and treat it as the easier half: 4x00 already gives you the phishing timeline, domains, and SPF/DKIM results; your job here is mostly selecting the exact citation-worthy facts and attaching ATT&CK IDs, not discovering anything new.
- For Stage 2, start from the 4x01 beacon-interval finding as your reference pattern, then go to the firewall sessions and look for sessions to the SAME C2 IP that match that interval — confirming the pattern independently is what upgrades this from single-source to CONVERGED.
- Specifically resolve the secondary-C2 timing question the task calls out: compare the first-seen timestamp for the unknown IP (from Task 3) against your Stage 2 timeline — did it appear in-window or after? State this as an explicit finding with a date, not a vague impression.
- Assign confidence level-by-level as you write, not as an afterthought pass at the end — it's much easier to get this right while the source-count for each specific claim is still fresh in your head.

Commands and patterns to reach for

```
# Find firewall sessions to the known C2 IP and compute inter-session intervals,
# to independently check the 5-minute beacon pattern 4x01 already found:
jq -r --arg ip "C2_IP" '.[.] | select(.dest_ip==$ip) | .timestamp' \
ir_evidence/firewall_sessions_ws_recv_03.json | sort | \
awk '{t=$0; "date -d \"\"t\"\" +%s" | getline e; if(p) print e-p; p=e}'

# Earliest appearance of the unknown/secondary IP, to settle the Stage-2-or-later question:
jq -r --arg ip "UNKNOWN_IP" '.[.] | select(.dest_ip==$ip) | .timestamp] | sort | first' \
ir_evidence/firewall_sessions_ws_recv_03.json
```

Watch out for

Don't let Stage 1 and Stage 2 blur together into one undifferentiated narrative. The task wants two clearly separated sections because they represent genuinely different attacker objectives (harvest a credential vs. establish durable remote access) and, often, a real time gap between them that itself is worth reporting as a

specific duration in your Stage 1–2 summary.

What this exercise is

The capstone narrative task for this batch (Stage 3 and Stage 5/6, if they exist in your full task list, aren't covered in this guide — see the note at the end). Stage 4 is the lateral-movement-and-staging phase your 4x04 threat hunt discovered behaviorally but couldn't fully detail. Here you fuse FOUR evidence sources — the 4x04 hunt findings, IR memory, IR disk, and IR firewall — into one pivot-by-pivot chain, and you explicitly answer the questions hunting alone left open: which credential, which hosts, what exactly was accessed, and how far staging actually got before containment.

What you should learn

- How to build a pivot chain as a literal sequence of (source host → tool → credential → target host → timestamp) tuples, each one individually cited, rather than one paragraph describing lateral movement in general terms.
- How IR evidence can reveal steps a SIEM-based hunt structurally couldn't see — this task explicitly asks you to look for lateral-movement steps the hunt MISSED but memory/firewall evidence reveals, which is the clearest illustration in the whole course of why hunting and IR are complementary, not redundant.
- How to sequence a data-flow question correctly (did the attacker stage directly on the database server, or pull data back to the pivot host first?) using timestamp ordering across sources, not assumption — the file-creation order in Task 2's NTFS timeline is your primary evidence here.
- How to write a counterfactual (“if not contained”) section responsibly: it must be grounded in a specific number (staging file sizes, session cadence) and labeled as an estimate, not presented as another confirmed finding.

How to approach it

- Build the pivot chain in strict chronological order, cross-checking each step against every source that could plausibly see it: a PsExec execution should show up in the 4x04 hunt output AND, often, as a corresponding session in the firewall logs, and the credential it used should trace back to the LSASS access you catalogued in Task 1.
- For the credential-access sub-section, don't just restate the 4x04 T1003.001 finding — the whole point of this task is asking whether IR memory evidence shows anything the hunt's SIEM-only view couldn't: a loaded module, a process handle, a second credential touched that the hunt never had visibility into.
- For data access and staging, lay Task 2's NTFS timeline directly against your pivot chain's timestamps — the ordering of “query result file created” vs. “lateral movement to database server” vs. “archive created” tells you definitively whether data was pulled TO the pivot host before compression, which is exactly the sequencing question the task asks you to resolve.
- Write the containment section last, once every other timestamp is settled — the exact moment the hunt's detection triggered needs to be your latest-confirmed event, with everything else in the chain falling before it.

Commands and patterns to reach for

```
# Cross-check a specific 4x04-hunt timestamp against firewall session data for the
# same host/port, to find corroborating (or missed) lateral-movement sessions:
jq -r --arg host "TARGET_HOST" '.[ ] | select(.dest_host==$host or .dest_ip=="TARGET_IP") | \'
' [.timestamp, .dest_port, .bytes_out, .bytes_in] | @tsv' \
ir_evidence/firewall_sessions_ws_recv_03.json | sort

# Order every file-system event from Task 2 against every network-session event from
# Task 3 on one shared timeline (merge, then sort on the first field):
```

```
cat task2_fs_events.tsv task3_sessions.tsv | sort -k1,1
```

```
# Simple duration estimate for the "if not contained" section (edit the numbers to  
# your own evidence): bytes remaining / average observed transfer rate:  
python3 -c "print(round(REMAINING_BYTES / (BYTES_SO_FAR / ELAPSED_SECONDS), 1))"
```

Watch out for

This task explicitly asks you to find lateral-movement steps the 4x04 hunt MISSED. Don't skip this just because it feels like it's second-guessing your own earlier work — a hunt built on SIEM/Sysmon telemetry alone has structural blind spots (e.g. connections that completed before log collection, or hosts outside the hunt's original scope), and finding one of those gaps is a genuine, valuable result, not a criticism of the original hunt.

What this exercise is

A pure merge-and-verify task: take the event lists your Stage 1–2 (Task 5), Stage 3 (Task 6 — see note below), and Stage 4 (Task 7) reconstructions already produced, and combine them into one single chronological sequence with no duplicate events, plus a set of computed temporal metrics (dwell time, breakout time, time to persistence, time to staging, detection-to-containment). This task doesn't introduce new evidence — it's where the three narratives you already wrote get stress-tested against each other.

What you should learn

- Why a unified timeline can surface an error that three separate stage narratives hide: the task gives the example of malware appearing to deploy BEFORE C2 is established, which is logically impossible for this attack chain — you only catch that kind of contradiction once everything sits on one sorted axis.
- How to turn a reconstructed narrative into real numbers: dwell time, breakout time, and operational tempo aren't qualitative judgments, they're arithmetic on two timestamps you already have — this is the same epoch-seconds-subtraction skill from 4x04, just applied across your whole reconstruction instead of one hunt.
- Why some events genuinely cannot be ordered relative to each other with confidence, and why saying so explicitly (rather than picking an arbitrary order) is the more rigorous choice.

How to approach it

- Extract each stage script's event list as structured rows (you'll want a consistent line format — timestamp / description / technique / confidence / sources — across Tasks 5, 6 and 7 specifically so this merge step is mechanical rather than a rewrite).
- Concatenate and sort by timestamp. Watch the boundary between stages carefully: the last event you cited at the end of Stage 2 and the first event at the start of Stage 3 may be the SAME underlying evidence cited from two angles — dedupe on that overlap rather than listing it twice.
- Compute each temporal metric as a direct epoch-seconds difference between two specific, named events in your own timeline (e.g. breakout time = timestamp of first lateral-movement event minus timestamp of initial-access event) — state which two events you used for each metric, so the number is reproducible, not just asserted.
- Do the gap-identification pass last: once the full sequence is sorted, look for any span of hours/days with zero events and ask whether that's a real dormant period (consistent with the attacker's known operational tempo) or a collection gap (no evidence source had visibility during that window).

Commands and patterns to reach for

```
# Merge three already-sorted event lists and re-sort on the timestamp field (col 1):
cat stage1_2_events.tsv stage3_events.tsv stage4_events.tsv | sort -t'\t' -k1,1

# Compute a temporal metric as a plain epoch-seconds delta between two named events:
T0=$(date -d "FIRST_ACCESS_TIMESTAMP" +%s)
T1=$(date -d "FIRST_LATERAL_MOVEMENT_TIMESTAMP" +%s)
echo "Breakout time: $(( (T1 - T0) / 3600 )) hours"

# Find gaps: compute the delta between every CONSECUTIVE pair of events in the merged,
# sorted list, and flag any delta above a threshold as a candidate gap:
awk -F'\t' '{cmd="date -d \""$1"\" +%s"; cmd | getline e; close(cmd); if (p) print e-p, $1;
```

```
p=e}' merged_timeline.tsv
```

Watch out for

Task 6 (Stage 3 Reconstruction) is a direct, named dependency here (“Merges the outputs of T5, T6 and T7”) but wasn’t included in the material provided for this guide. From the report structure in Task 15, Stage 3 is almost certainly “Malware Deployment” — built the same way as Tasks 5 and 7, using 4x03’s malware findings plus the IR memory evidence confirming the RAT/dropper process was actually running. You’ll need that task’s real instructions before Task 8 can be complete; this guide can be extended to cover it if you paste its description.

What this exercise is

The definitive, whole-project re-assessment of every ATT&CK technique in the threat model, using `reference/attck_navigator_80pct.json` as your starting point. Unlike every earlier project's ATT&CK work (which assessed techniques using whatever one investigation had just produced), this task re-evaluates EVERY technique against the FULL evidence picture you now have from Tasks 0–8, then adds five explicitly-named new techniques the IR evidence introduced, and computes the final coverage percentage.

What you should learn

- That technique re-assessment is not a one-directional ratchet. The task explicitly asks you to consider CORRECTED and DOWNGRADED outcomes, not just upgrades — if IR evidence contradicts an earlier CONFIRMED finding, the honest move is to say so, even though it feels like undoing earlier work.
- How a single piece of new evidence can resolve an old INFERRED call: this is the same upgrade logic you already used once in 4x04 (Task 11 there upgraded 5 techniques from NOT COVERED to OBSERVED using hunt evidence) — here you're doing one more pass, informed by IR evidence the hunt never had.
- Why 100% coverage is not the goal and should make you suspicious if you reach it. The expected output explicitly leaves 1 technique as a genuine gap — learn to distinguish “we have no evidence either way” (a real, honestly-reported limitation) from “we're confident this wasn't used” (also a legitimate finding, but a different one).

How to approach it

- Go through the baseline JSON technique-by-technique, not evidence-source-by-source — for each technique already present, ask “did anything in Tasks 0–8 add evidence for or against this specific technique?” before moving to the next one. This is a mechanical pass; resist the temptation to batch-upgrade techniques that merely seem related to things you found.
- Add the five explicitly-listed new techniques (T1053.005, T1074.001, T1560.001, T1070.001, T1005) using the exact evidence citations you already built in Tasks 1 and 2 for each one — you did the underlying analysis already, this task is mostly about formally slotting it into the ATT&CK structure (tactic, confidence, first-identified-by).
- Compute coverage as a straight fraction (confirmed-or-better techniques ÷ total techniques in the threat model) and show your work, same as you'll have done in Task 8's metrics — don't just assert a percentage.
- Write the “remaining gap” section with real intellectual honesty: state explicitly whether the gap is a collection limitation (no source could have seen it) or a genuine absence (you have positive reason to believe the technique wasn't used).

Commands and patterns to reach for

```
# Re-usable pattern from 4x04 for upgrading a technique's score/color/comment in place
# (adjust the technique ID list and comments to your own re-assessment):
jq --argjson ids '["T1053.005","T1074.001","T1560.001","T1070.001","T1005"]' '
.techniques += [$ids[] | {techniqueID: ., score: 3, color: "#c40000", enabled: true}]
' reference/attck_navigator_80pct.json > attack_layer_final.json

# Compute the final coverage percentage directly from the technique array:
jq ' [.techniques[] | select(.score==3)] | length' attack_layer_final.json
jq '.techniques | length' attack_layer_final.json
```

Watch out for

Before trusting any baseline percentage from an earlier project's own stored summary, recompute it directly from that file's technique array. This exact caution saved real work in the 4x04 project (the reference ATT&CK file's own stored "16 observed" summary didn't match a direct count of its technique scores, which turned out to be 14) — always verify a baseline number before building on it rather than trusting a label.

What this exercise is

The business question everything else in this project exists to answer: was patient data compromised, and does it cross the HIPAA breach-notification threshold? You map every compromised host to reference/meddefense_asset_inventory.txt to see what data it holds, classify each host's access level (confirmed / potential / no access), settle the exfiltration question by comparing Task 2's staging totals against Task 3's suspicious-outbound totals, and combine both into a per-data-type exposure table with a regulatory determination.

What you should learn

- Why “data was accessed” and “data was exfiltrated” are legally distinct findings with different notification consequences — and why a reconstruction that blurs them is not just imprecise, it's potentially legally consequential.
- How to apply a three-tier access model (CONFIRMED / POTENTIAL / NO ACCESS) to systems rather than to individual claims — CONFIRMED needs a specific interaction artifact (e.g. a file creation timestamp matching the attacker's window), POTENTIAL is reachability-plus-credentials without interaction evidence, and conflating the two is exactly the error that makes a breach assessment indefensible.
- How a byte-count comparison you already half-built in Task 3 becomes the deciding fact for the whole exfiltration question — this task doesn't ask you to re-derive that number, it asks you to use it as the hinge of a legal/regulatory conclusion.

How to approach it

- Build the host-to-data mapping first, as a simple lookup: take every host that appeared anywhere in your Task 7 lateral-movement chain and join it against the asset inventory's data-sensitivity classification. This part is mechanical.
- Settle access level per host using a strict evidentiary rule, not impression: did you cite a specific artifact in Task 7 showing the attacker interacted with data on this host (confirmed), or only that they had the credential and network path to reach it (potential)? Hosts never in the lateral-movement chain at all are simply no access.
- Resolve the exfiltration question by reusing Task 3's own comparison (suspicious outbound bytes vs. Task 2's staging file sizes) — don't recompute this from scratch, cite it. State explicitly whether the comparison shows transmission occurred, was partial, or didn't happen at all, and WHY (what interrupted it, with a timestamp).
- Only after access level and exfiltration status are both settled, build the per-data-type table (PHI / insurance / employee / operational) — each row's status should follow directly and traceably from the host-level findings above it, not be assigned independently.
- Write the regulatory section as a direct syllogism: here is the evidence, here is the HIPAA threshold language, here is whether the evidence meets it, here are the mitigating factors (interruption timing, any encryption, access controls) that bear on severity even if the threshold is met.

Commands and patterns to reach for

```
# Join your lateral-movement host list against the asset inventory's data classification:
grep -F -f compromised_hosts.txt reference/meddefense_asset_inventory.txt
```

```
# Reuse (don't recompute) Task 3's exfiltration comparison:
STAGED_BYTES=36400000 # from Task 2, in bytes
```

```
OUTBOUND_BYTES=$(jq '[.[] | select(.dest_ip=="SUSPICIOUS_IP") | .bytes_out] | add' \
ir_evidence/firewall_sessions_ws_recv_03.json)
echo "Staged: $STAGED_BYTES Transmitted to suspicious dest: ${OUTBOUND_BYTES:-0}"
```

Watch out for

Resist rounding “potential access” up to “confirmed” just because it feels more conclusive, and resist rounding “confirmed” down to “potential” to be conservative. Both directions are dishonest in the same way: the regulatory and board audiences explicitly named in this project need the real evidentiary line, not a softened or inflated one — that’s the entire point of building a three-tier model instead of a binary yes/no.

What this exercise is

The final deliverable — a Markdown report, not a script — that synthesizes every task from T0 through T14 into one document serving two audiences at once: the SOC team (technical detail, ATT&CK mappings, detection specifics) and the board (clear narrative, impact, remediation timeline). This is the same “two-audience” discipline every capstone report in this course has required, now applied at the scale of the entire campaign rather than one investigation.

What you should learn

- How to write a synthesis document that doesn't re-derive analysis — every section of this report should point back to a task you already completed and compress its finding into report prose, not redo the work inline.
- How to write an executive summary that survives scrutiny: one paragraph covering what happened, how far the attacker got, how it was stopped, what's next, and the headline metrics — dense, but every sentence traceable to a specific task's output.
- Why the conclusions section asks you to reflect on the course's own thesis (“why investigation in pieces creates blind spots that only reconstruction reveals”) — this is the moment to articulate, in your own words, the thing this entire six-project arc was teaching.

How to approach it

- Build this task LAST, after every other task's output genuinely exists — it is explicitly a synthesis of T0 through T14, and several of its required sections (ATT&CK Navigator layer reference from T10, Defensive Posture from T13, Remediation Plan from T14) depend on tasks not covered anywhere in this guide yet.
- Work through the report's required section list as a checklist against your own completed tasks: Executive Summary, Methodology (from T0), Attack Reconstruction (T5–T8), Unified Timeline (T8), ATT&CK Analysis (T9–T11), Impact Assessment (T12), Defensive Posture (T13), Remediation Plan (T14), Conclusions, Appendices. If a section's source task doesn't exist yet, that section can't be written honestly yet.
- For the appendices (IOC summary table, evidence citation index, ATT&CK layer reference), these are almost direct exports of work you've already built — Task 4's IOC correlation matrix and Task 9's technique inventory are your primary source material, reformatted for an appendix rather than re-analyzed.
- Write the executive summary LAST, even though it appears first in the document — it's much easier to compress an already-complete report into one tight paragraph than to write the summary first and then make the rest of the report match it.

Watch out for

This task has real, named dependencies that are not yet covered in this guide: Task 6 (Stage 3), Task 10 (likely the ATT&CK Navigator layer file itself, referenced as an appendix), Task 11 (unclear from context — possibly a gap/blind-spot analysis, given Task 15 cites a “Gap analysis and blind spot assessment” under ATT&CK Analysis), Task 13 (Defensive Posture Evaluation), and Task 14 (Remediation Plan). Don't start Task 15 until you have those five tasks' real instructions and outputs — send them over and this guide can be extended before you need them.

A Note on What's Not In This Guide

This guide covers Tasks 0, 1, 2, 3, 4, 5, 7, 8, 9, 12 and 15 — eleven tasks, all confirmed as the full set of materials provided when this guide was built. Five task numbers are known gaps, each one a confirmed, named dependency of a task covered here rather than a guess: **Task 6** (Stage 3 Reconstruction — a direct dependency of Task 8's merge and almost certainly “Malware Deployment,” inferred from Task 15's report structure), **Task 10** (ATT&CK Navigator layer file, referenced as a Task 15 appendix), **Task 11** (likely a gap/blind-spot analysis, referenced under Task 15's “ATT&CK Analysis” section), **Task 13** (Defensive Posture Evaluation), and **Task 14** (Remediation Plan). If you have those five task descriptions, send them over and this guide can be extended to cover the complete project.

One last piece of advice that applies to every task above: build your evidence citations as you go, not at the end. By the time you reach Task 7 or Task 8 you'll be drawing on six or more prior outputs at once, and a habit of writing “[source file / field / timestamp]” next to every fact the moment you find it — rather than trying to reconstruct where a fact came from after the fact — is the single thing most likely to save you time across this whole project.